

1. What We Have Already Covered

We have already covered the major joins:

INNER JOIN
LEFT JOIN
RIGHT JOIN

Now we will cover the remaining joins that are important for interviews and practical SQL understanding:

FULL OUTER JOIN
CROSS JOIN
SELF JOIN
NON-EQUI JOIN
SEMI JOIN
ANTI JOIN
NATURAL JOIN

The focus is not only syntax.

For every join, understand:

Scenario
Why this join is needed
Query
Expected output logic
Interview point

2. Database Setup

Run this first.

```
DROP DATABASE IF EXISTS advanced_joins_practice;
```

```
CREATE DATABASE advanced_joins_practice;
```

USE advanced_joins_practice;

3. Create Tables

customers table

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(50),  
    city VARCHAR(50)  
);
```

orders table

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_amount DECIMAL(10,2),  
    order_date DATE,  
    order_status VARCHAR(30)  
);
```

employees table

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(50),  
    manager_id INT,  
    department VARCHAR(50),  
    salary DECIMAL(10,2)  
);
```

salary_bands table

```
CREATE TABLE salary_bands (  
    band_name VARCHAR(20),  
    min_salary DECIMAL(10,2),  
    max_salary DECIMAL(10,2)  
);
```

products table

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(50),  
    category VARCHAR(50)  
);
```

stores table

```
CREATE TABLE stores (  
    store_id INT PRIMARY KEY,  
    store_name VARCHAR(50),  
    city VARCHAR(50)  
);
```

source_customers table

This table represents customer data from the old/source system.

```
CREATE TABLE source_customers (  
    customer_id INT,  
    customer_name VARCHAR(50),  
    city VARCHAR(50)  
);
```

target_customers table

This table represents customer data loaded into the new/target system.

```
CREATE TABLE target_customers (  
  customer_id INT,  
  customer_name VARCHAR(50),  
  city VARCHAR(50)  
);
```

4. Insert Sample Data

Insert customers

```
INSERT INTO customers  
(customer_id, customer_name, city)  
VALUES  
(1, 'Anurag', 'Bengaluru'),  
(2, 'Stuti', 'Pune'),  
(3, 'Rahul', 'Delhi'),  
(4, 'Priya', 'Mumbai'),  
(5, 'Neha', 'Bengaluru');
```

Insert orders

```
INSERT INTO orders  
(order_id, customer_id, order_amount, order_date, order_status)  
VALUES  
(101, 1, 50000.00, '2026-06-01', 'Delivered'),  
(102, 1, 1000.00, '2026-06-02', 'Delivered'),  
(103, 2, 2500.00, '2026-06-03', 'Delivered'),  
(104, 99, 15000.00, '2026-06-04', 'Delivered'),  
(105, NULL, 45000.00, '2026-06-05', 'Pending');
```

Data observation:

Customer 1 has two orders.
Customer 2 has one order.
Customers 3, 4, and 5 have no orders.
Order 104 has invalid customer_id 99.
Order 105 has NULL customer_id.

Insert employees

```
INSERT INTO employees
(employee_id, employee_name, manager_id, department, salary)
VALUES
(1, 'Anurag', NULL, 'Engineering', 200000.00),
(2, 'Stuti', 1, 'Engineering', 120000.00),
(3, 'Shalini', 1, 'Engineering', 130000.00),
(4, 'Vaishnav', 2, 'Engineering', 90000.00),
(5, 'Shruti', 2, 'Engineering', 85000.00),
(6, 'Ritika', NULL, 'HR', 150000.00),
(7, 'Stuti', 6, 'HR', 70000.00);
```

Data observation:

Anurag is the Engineering head.
Engineering Stuti and Shalini report to Anurag.
Vaishnav and Shruti report to Engineering Stuti.
Ritika is the HR head.
HR Stuti reports to Ritika.

Note:

There are two employees named Stuti.
Use employee_id or department where needed to avoid confusion.

Insert salary bands

```
INSERT INTO salary_bands
(band_name, min_salary, max_salary)
VALUES
('Junior', 0.00, 80000.00),
('Mid', 80001.00, 120000.00),
('Senior', 120001.00, 160000.00),
('Lead', 160001.00, 250000.00);
```

Insert products

```
INSERT INTO products
(product_id, product_name, category)
VALUES
(201, 'Laptop', 'Electronics'),
(202, 'Mouse', 'Accessories'),
(203, 'Keyboard', 'Accessories');
```

Insert stores

```
INSERT INTO stores
(store_id, store_name, city)
VALUES
(1, 'Bengaluru Store', 'Bengaluru'),
(2, 'Pune Store', 'Pune'),
(3, 'Delhi Store', 'Delhi');
```

Insert source customers

```
INSERT INTO source_customers
(customer_id, customer_name, city)
VALUES
(1, 'Anurag', 'Bengaluru'),
(2, 'Stuti', 'Pune'),
(3, 'Rahul', 'Delhi'),
(4, 'Priya', 'Mumbai');
```

Insert target customers

```
INSERT INTO target_customers
(customer_id, customer_name, city)
VALUES
(1, 'Anurag', 'Bengaluru'),
(2, 'Stuti', 'Pune'),
(3, 'Rahul', 'Noida'),
(5, 'Neha', 'Bengaluru');
```

Data observation:

Customer 1 matches in both source and target.

Customer 2 matches in both source and target.

Customer 3 exists in both, but city is different.

Customer 4 exists only in source.

Customer 5 exists only in target.

5. Check the Data

```
SELECT * FROM customers;
```

```
SELECT * FROM orders;
```

```
SELECT * FROM employees;
```

```
SELECT * FROM salary_bands;
```

```
SELECT * FROM products;
```

```
SELECT * FROM stores;
```

```
SELECT * FROM source_customers;
```

```
SELECT * FROM target_customers;
```

6. FULL OUTER JOIN

What is FULL OUTER JOIN?

FULL OUTER JOIN returns:

Matching records from both tables

+

Unmatched records from left table

+

Unmatched records from right table

Simple meaning:

FULL OUTER JOIN = complete data from both sides

Scenario

A company migrated customer data from a source system to a target system.

Now we want to check:

Which records matched?

Which records are missing in target?

Which records are extra in target?

Which records have mismatched values?

Why FULL OUTER JOIN?

Because we need complete comparison from both source and target.

- INNER JOIN will show only matching records.
 - LEFT JOIN will help find records missing in target.
 - RIGHT JOIN will help find extra records in target.
 - FULL OUTER JOIN gives complete visibility from both sides.
-

Query for Databricks / Snowflake / PostgreSQL / SQL Server

```
SELECT
  COALESCE(s.customer_id, t.customer_id) AS customer_id,
  s.customer_name AS source_name,
  t.customer_name AS target_name,
  s.city AS source_city,
  t.city AS target_city,
  CASE
    WHEN t.customer_id IS NULL THEN 'Missing in Target'
    WHEN s.customer_id IS NULL THEN 'Extra in Target'
    WHEN s.city <> t.city THEN 'City Mismatch'
    ELSE 'Matched'
  END AS validation_status
FROM source_customers s
FULL OUTER JOIN target_customers t
  ON s.customer_id = t.customer_id;
```


Expected logic:

Customer 1 → Matched
Customer 2 → Matched
Customer 3 → City Mismatch
Customer 4 → Missing in Target
Customer 5 → Extra in Target

MySQL Version

MySQL does not directly support FULL OUTER JOIN.

So we simulate it using:

LEFT JOIN + UNION + RIGHT JOIN

```
SELECT
  COALESCE(s.customer_id, t.customer_id) AS customer_id,
  s.customer_name AS source_name,
  t.customer_name AS target_name,
  s.city AS source_city,
  t.city AS target_city,
  CASE
    WHEN t.customer_id IS NULL THEN 'Missing in Target'
    WHEN s.customer_id IS NULL THEN 'Extra in Target'
    WHEN s.city <> t.city THEN 'City Mismatch'
    ELSE 'Matched'
  END AS validation_status
FROM source_customers s
LEFT JOIN target_customers t
  ON s.customer_id = t.customer_id
```

UNION

```
SELECT
  COALESCE(s.customer_id, t.customer_id) AS customer_id,
  s.customer_name AS source_name,
  t.customer_name AS target_name,
  s.city AS source_city,
  t.city AS target_city,
  CASE
    WHEN t.customer_id IS NULL THEN 'Missing in Target'
```

```
        WHEN s.customer_id IS NULL THEN 'Extra in Target'
        WHEN s.city <> t.city THEN 'City Mismatch'
        ELSE 'Matched'
    END AS validation_status
FROM source_customers s
RIGHT JOIN target_customers t
    ON s.customer_id = t.customer_id;
```

Practice Question 1

Find customers present in source but missing in target.

Solution

```
SELECT
    s.customer_id,
    s.customer_name,
    s.city
FROM source_customers s
LEFT JOIN target_customers t
    ON s.customer_id = t.customer_id
WHERE t.customer_id IS NULL;
```

Expected result:

Priya

Practice Question 2

Find customers present in target but missing in source.

Solution

```
SELECT
    t.customer_id,
    t.customer_name,
    t.city
FROM target_customers t
LEFT JOIN source_customers s
```

```
ON t.customer_id = s.customer_id  
WHERE s.customer_id IS NULL;
```

Expected result:

Neha

Practice Question 3

Find customers where city is different between source and target.

Solution

```
SELECT  
    s.customer_id,  
    s.customer_name,  
    s.city AS source_city,  
    t.city AS target_city  
FROM source_customers s  
INNER JOIN target_customers t  
    ON s.customer_id = t.customer_id  
WHERE s.city <> t.city;
```

Expected result:

Rahul → Delhi in source, Noida in target

7. CROSS JOIN

What is CROSS JOIN?

CROSS JOIN returns all possible combinations between two tables.

If one table has 3 rows and another table has 3 rows:

Output = 3 x 3 = 9 rows

Scenario

Business wants to create a product-store planning template.

They want every product mapped with every store.

Example:

Laptop - Bengaluru Store

Laptop - Pune Store

Laptop - Delhi Store

Mouse - Bengaluru Store

Mouse - Pune Store

Mouse - Delhi Store

Why CROSS JOIN?

Because we need every product with every store.

Query

```
SELECT
  p.product_id,
  p.product_name,
  s.store_id,
  s.store_name
FROM products p
CROSS JOIN stores s;
```

Expected count:

products = 3

stores = 3

output = 9 rows

Count Check

```
SELECT COUNT(*) AS total_combinations
FROM products p
CROSS JOIN stores s;
```

Why this query?

Before using CROSS JOIN, always check expected row count.

CROSS JOIN multiplies rows.

Practice Question 4

Create all product and store combinations.

Solution

```
SELECT
  p.product_name,
  s.store_name
FROM products p
CROSS JOIN stores s;
```

Practice Question 5

Find total number of product-store combinations.

Solution

```
SELECT COUNT(*) AS total_product_store_combinations
FROM products p
CROSS JOIN stores s;
```

Expected result:

9

Interview Point

CROSS JOIN is useful for creating combinations.

But it can be dangerous on large data.

Example:

1 lakh rows x 1 lakh rows = 10,000,000,000 rows

So never use CROSS JOIN without understanding row count.

8. SELF JOIN

What is SELF JOIN?

SELF JOIN means joining a table with itself.

There is no separate **SELF JOIN** keyword.

We use normal JOIN, but the same table is used twice with different aliases.

Scenario

HR wants to see every employee with their manager name.

In the employees table:

employee_id = employee's own ID

manager_id = manager's employee_id

The manager is also an employee.

So both employee and manager data are stored in the same table.

Why SELF JOIN?

Because we need to join the employees table with itself.

Query: Employee With Manager Name

```
SELECT
  e.employee_id,
  e.employee_name AS employee_name,
  e.department AS employee_department,
  m.employee_name AS manager_name,
  m.department AS manager_department
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

Why LEFT JOIN?

Because top-level managers like Anurag and Ritika do not have managers.

If we use INNER JOIN, top-level managers will disappear.

Expected logic:

Anurag → No Manager
Engineering Stuti → Anurag
Shalini → Anurag
Vaishnav → Engineering Stuti
Shruti → Engineering Stuti
Ritika → No Manager
HR Stuti → Ritika

Practice Question 6

Show employees with their manager names.

Solution

```
SELECT
  e.employee_name AS employee_name,
  e.department AS employee_department,
  m.employee_name AS manager_name,
```

```
m.department AS manager_department
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

Practice Question 7

Find employees who do not have a manager.

Scenario

HR wants to identify top-level employees.

Solution

```
SELECT
  e.employee_id,
  e.employee_name,
  e.department
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id
WHERE m.employee_id IS NULL;
```

Expected result:

Anurag
Ritika

Practice Question 8

Find employees directly reporting to Anurag.

Scenario

Anurag wants to see his direct reportees.

Solution

```
SELECT
```



```
e.employee_id,  
e.employee_name,  
e.department  
FROM employees e  
INNER JOIN employees m  
  ON e.manager_id = m.employee_id  
WHERE m.employee_name = 'Anurag';
```

Expected result:

Engineering Stuti
Shalini

Practice Question 9

Find employees directly reporting to Engineering Stuti.

Scenario

There are two employees named Stuti, so we should filter using manager employee ID.

Engineering Stuti has employee_id = 2.

Solution

```
SELECT  
  e.employee_id,  
  e.employee_name,  
  e.department  
FROM employees e  
INNER JOIN employees m  
  ON e.manager_id = m.employee_id  
WHERE m.employee_id = 2;
```

Expected result:

Vaishnav
Shruti

Interview Point

SELF JOIN is commonly used for hierarchy problems.

Examples:

Employee-manager hierarchy
Category-subcategory hierarchy
Parent-child relationship
Comparing rows in the same table

9. NON-EQUI JOIN / RANGE JOIN

What is NON-EQUI JOIN?

Most joins use equality:

ON a.id = b.id

NON-EQUI JOIN uses conditions other than equality.

Examples:

BETWEEN

>

<

>=

<=

Scenario

HR wants to classify each employee into a salary band.

Salary bands are stored as ranges:

Junior: 0 to 80000

Mid: 80001 to 120000

Senior: 120001 to 160000
Lead: 160001 to 250000

Employee salary will not exactly equal the band name.

It falls between min and max salary.

Why NON-EQUI JOIN?

Because the join condition is based on a range, not exact equality.

Query

```
SELECT
  e.employee_id,
  e.employee_name,
  e.department,
  e.salary,
  sb.band_name
FROM employees e
INNER JOIN salary_bands sb
  ON e.salary BETWEEN sb.min_salary AND sb.max_salary;
```

Expected logic:

Anurag → Lead
Engineering Stuti → Mid
Shalini → Senior
Vaishnav → Mid
Shruti → Mid
Ritika → Senior
HR Stuti → Junior

Practice Question 10

Assign salary band to each employee.

Solution

```
SELECT
    e.employee_name,
    e.department,
    e.salary,
    sb.band_name
FROM employees e
INNER JOIN salary_bands sb
    ON e.salary BETWEEN sb.min_salary AND sb.max_salary;
```

Practice Question 11

Find only Senior and Lead employees.

Scenario

HR wants employees in higher salary bands.

Solution

```
SELECT
    e.employee_name,
    e.department,
    e.salary,
    sb.band_name
FROM employees e
INNER JOIN salary_bands sb
    ON e.salary BETWEEN sb.min_salary AND sb.max_salary
WHERE sb.band_name IN ('Senior', 'Lead');
```

Expected result:

Anurag
Shalini
Ritika

Interview Point

NON-EQUI JOIN is used for range-based mapping.

Common examples:

Salary bands

Tax slabs

Commission slabs

Date ranges

SCD Type 2 effective date joins

Age groups

10. SEMI JOIN

What is SEMI JOIN?

SEMI JOIN returns rows from the first table where a match exists in the second table.

It returns only columns from the first table.

Simple meaning:

Give me records from table A where matching records exist in table B.

Scenario

Marketing team wants customers who have placed at least one order.

They do not need order details.

They only need customer details.

Why SEMI JOIN?

Because we only need to check whether an order exists or not.

We do not need columns from the orders table.

MySQL Query Using EXISTS

```
SELECT
  c.customer_id,
  c.customer_name,
  c.city
FROM customers c
WHERE EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.customer_id
);
```

Expected result:

Anurag
Stuti

Same Logic Using INNER JOIN + DISTINCT

```
SELECT DISTINCT
  c.customer_id,
  c.customer_name,
  c.city
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id;
```

Why EXISTS is better here?

Because we only need customer records.

INNER JOIN can create duplicates if one customer has multiple orders, so we may need DISTINCT.

Databricks / Spark SQL Version

```
SELECT
  c.customer_id,
```

```
c.customer_name,  
c.city  
FROM customers c  
LEFT SEMI JOIN orders o  
ON c.customer_id = o.customer_id;
```

Practice Question 12

Find customers who placed at least one order.

Solution using EXISTS

```
SELECT  
  c.customer_id,  
  c.customer_name,  
  c.city  
FROM customers c  
WHERE EXISTS (  
  SELECT 1  
  FROM orders o  
  WHERE o.customer_id = c.customer_id  
);
```

Practice Question 13

Find employees who are managers of at least one employee.

Scenario

HR wants a list of employees who have at least one direct reportee.

Solution

```
SELECT  
  m.employee_id,  
  m.employee_name,  
  m.department  
FROM employees m  
WHERE EXISTS (  
  SELECT 1
```

```
FROM employees e
WHERE e.manager_id = m.employee_id
);
```

Expected result:

Anurag
Engineering Stuti
Ritika

Interview Point

SEMI JOIN checks existence.

It does not return columns from the second table.

If we need columns from both tables, use INNER JOIN.

11. ANTI JOIN

What is ANTI JOIN?

ANTI JOIN returns rows from the first table where no match exists in the second table.

Simple meaning:

Give me records from table A that do not exist in table B.

Scenario

Marketing team wants customers who signed up but never placed any order.

Why ANTI JOIN?

Because we want customers for whom no matching order exists.

MySQL Query Using NOT EXISTS

```
SELECT
  c.customer_id,
  c.customer_name,
  c.city
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.customer_id
);
```

Expected result:

Rahul
Priya
Neha

MySQL Query Using LEFT JOIN

```
SELECT
  c.customer_id,
  c.customer_name,
  c.city
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

This gives the same result.

Databricks / Spark SQL Version

```
SELECT
```

```
c.customer_id,  
c.customer_name,  
c.city  
FROM customers c  
LEFT ANTI JOIN orders o  
ON c.customer_id = o.customer_id;
```

Practice Question 14

Find customers who never placed any order.

Solution using NOT EXISTS

```
SELECT  
    c.customer_id,  
    c.customer_name,  
    c.city  
FROM customers c  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM orders o  
    WHERE o.customer_id = c.customer_id  
);
```

Practice Question 15

Find employees who are not managers.

Scenario

HR wants employees who do not have any reportees.

Solution

```
SELECT  
    e.employee_id,  
    e.employee_name,  
    e.department  
FROM employees e  
WHERE NOT EXISTS (
```

```
SELECT 1
FROM employees r
WHERE r.manager_id = e.employee_id
);
```

Expected result:

Shalini
Vaishnav
Shruti
HR Stuti

NOT IN Trap

Some students write:

```
SELECT
    c.customer_id,
    c.customer_name
FROM customers c
WHERE c.customer_id NOT IN (
    SELECT customer_id
    FROM orders
);
```

This can give wrong or empty output if the subquery contains NULL.

Our orders table has `customer_id = NULL`.

Because of this, `NOT IN` can behave unexpectedly.

Safer options:

NOT EXISTS
LEFT JOIN + IS NULL
LEFT ANTI JOIN in Spark / Databricks

Interview Point

ANTI JOIN is mainly used to find missing or unmatched records.

Common examples:

Customers with no orders

Products never sold

Source records missing in target

Invalid records

Incremental load comparison

12. NATURAL JOIN

What is NATURAL JOIN?

NATURAL JOIN automatically joins tables based on columns with the same name.

Example:

Both `customers` and `orders` have:

`customer_id`

So NATURAL JOIN will automatically join using `customer_id`.

Scenario

A beginner wants to quickly join customers and orders because both tables have the same column name `customer_id`.

Why NATURAL JOIN?

Technically, NATURAL JOIN can do this automatically.

Query

```
SELECT *  
FROM customers  
NATURAL JOIN orders;
```

Why NATURAL JOIN Is Risky

NATURAL JOIN is usually avoided in production.

Reasons:

Join columns are selected automatically.

If a new same-name column is added, query behavior can change.

The join condition is not clearly visible.

Debugging becomes difficult.

It can create unexpected results.

Preferred Query

```
SELECT  
    c.customer_id,  
    c.customer_name,  
    c.city,  
    o.order_id,  
    o.order_amount,  
    o.order_status  
FROM customers c  
INNER JOIN orders o  
    ON c.customer_id = o.customer_id;
```

Why this is better?

Because the join condition is clearly visible.

Practice Question 16

Write a NATURAL JOIN between customers and orders.

Solution

```
SELECT *  
FROM customers  
NATURAL JOIN orders;
```

Better Production Version

```
SELECT  
    c.customer_id,  
    c.customer_name,  
    c.city,  
    o.order_id,  
    o.order_amount,  
    o.order_status  
FROM customers c  
INNER JOIN orders o  
    ON c.customer_id = o.customer_id;
```

Interview Point

NATURAL JOIN is mostly asked to check awareness.

In real projects, prefer explicit JOIN with ON condition.

13. Level-Wise Practice Questions

Level 1: Basic

Q1. Create all product-store combinations.

```
SELECT  
    p.product_name,  
    s.store_name  
FROM products p  
CROSS JOIN stores s;
```

Q2. Show every employee with manager name.

```
SELECT
    e.employee_name AS employee_name,
    e.department AS employee_department,
    m.employee_name AS manager_name
FROM employees e
LEFT JOIN employees m
    ON e.manager_id = m.employee_id;
```

Q3. Assign salary band to each employee.

```
SELECT
    e.employee_name,
    e.salary,
    sb.band_name
FROM employees e
INNER JOIN salary_bands sb
    ON e.salary BETWEEN sb.min_salary AND sb.max_salary;
```

Level 2: Intermediate**Q4. Find source customers missing in target.**

```
SELECT
    s.customer_id,
    s.customer_name,
    s.city
FROM source_customers s
LEFT JOIN target_customers t
    ON s.customer_id = t.customer_id
WHERE t.customer_id IS NULL;
```

Q5. Find target customers missing in source.

```
SELECT
    t.customer_id,
    t.customer_name,
```

```
t.city
FROM target_customers t
LEFT JOIN source_customers s
  ON t.customer_id = s.customer_id
WHERE s.customer_id IS NULL;
```

Q6. Find employees who are managers.

```
SELECT
  m.employee_id,
  m.employee_name,
  m.department
FROM employees m
WHERE EXISTS (
  SELECT 1
  FROM employees e
  WHERE e.manager_id = m.employee_id
);
```

Q7. Find employees who are not managers.

```
SELECT
  e.employee_id,
  e.employee_name,
  e.department
FROM employees e
WHERE NOT EXISTS (
  SELECT 1
  FROM employees r
  WHERE r.manager_id = e.employee_id
);
```

Level 3: Interview Level

Q8. Create a migration validation report.

```
SELECT
  COALESCE(s.customer_id, t.customer_id) AS customer_id,
  s.customer_name AS source_name,
```



```

t.customer_name AS target_name,
s.city AS source_city,
t.city AS target_city,
CASE
    WHEN t.customer_id IS NULL THEN 'Missing in Target'
    WHEN s.customer_id IS NULL THEN 'Extra in Target'
    WHEN s.city <> t.city THEN 'City Mismatch'
    ELSE 'Matched'
END AS validation_status
FROM source_customers s
LEFT JOIN target_customers t
    ON s.customer_id = t.customer_id

UNION

SELECT
    COALESCE(s.customer_id, t.customer_id) AS customer_id,
    s.customer_name AS source_name,
    t.customer_name AS target_name,
    s.city AS source_city,
    t.city AS target_city,
    CASE
        WHEN t.customer_id IS NULL THEN 'Missing in Target'
        WHEN s.customer_id IS NULL THEN 'Extra in Target'
        WHEN s.city <> t.city THEN 'City Mismatch'
        ELSE 'Matched'
    END AS validation_status
FROM source_customers s
RIGHT JOIN target_customers t
    ON s.customer_id = t.customer_id;

```

Q9. Find customers who ordered using SEMI JOIN logic.

```

SELECT
    c.customer_id,
    c.customer_name,
    c.city
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);

```

Q10. Find customers who never ordered using ANTI JOIN logic.

```
SELECT
  c.customer_id,
  c.customer_name,
  c.city
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.customer_id
);
```

Q11. Explain why this query can be risky.

```
SELECT
  c.customer_id,
  c.customer_name
FROM customers c
WHERE c.customer_id NOT IN (
  SELECT customer_id
  FROM orders
);
```

Answer:

This query can give unexpected results because the subquery contains **NULL**.

When **NOT IN** has NULL in the subquery result, many SQL databases may return no rows or incorrect results.

Better use:

```
SELECT
  c.customer_id,
  c.customer_name
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
```

```
WHERE o.customer_id = c.customer_id  
);
```

14. Interview Questions and Answers

Q1. What is FULL OUTER JOIN?

FULL OUTER JOIN returns matching and non-matching records from both tables.

It gives complete data from both sides.

Q2. Where is FULL OUTER JOIN used?

It is used in:

Source vs target comparison

Migration validation

Reconciliation

Audit checks

Finding missing records from both sides

Q3. Does MySQL support FULL OUTER JOIN?

No.

MySQL does not directly support FULL OUTER JOIN.

We can simulate it using:

LEFT JOIN + UNION + RIGHT JOIN

Q4. What is CROSS JOIN?

CROSS JOIN returns all possible combinations between two tables.

If table A has 3 rows and table B has 3 rows, output will be 9 rows.

Q5. When is CROSS JOIN useful?

It is useful when we need all combinations.

Examples:

Product-store combinations

Date-product combinations

Test scenario generation

Planning templates

Q6. Why can CROSS JOIN be dangerous?

Because it multiplies row counts.

Large tables can produce huge output and performance issues.

Q7. What is SELF JOIN?

SELF JOIN means joining a table with itself.

It is commonly used for employee-manager hierarchy or parent-child relationships.

Q8. Is SELF JOIN a separate SQL keyword?

No.

SELF JOIN is not a separate keyword.

We use normal JOIN with the same table and different aliases.

Q9. Why do we use LEFT JOIN in employee-manager SELF JOIN?

Because top-level employees may not have a manager.

If we use INNER JOIN, top-level employees will disappear.

Q10. What is NON-EQUI JOIN?

NON-EQUI JOIN uses conditions other than equality.

Examples:

BETWEEN

>

<

>=

<=

Q11. Where is NON-EQUI JOIN used?

It is used in:

Salary bands

Tax slabs

Commission slabs

Age groups

Date ranges

SCD Type 2 effective date joins

Q12. What is SEMI JOIN?

SEMI JOIN returns rows from the first table where a match exists in the second table.

It only returns columns from the first table.

Q13. What is ANTI JOIN?

ANTI JOIN returns rows from the first table where no match exists in the second table.

It is used to find missing or unmatched records.

Q14. Difference between SEMI JOIN and INNER JOIN?

INNER JOIN returns columns from both tables.

SEMI JOIN only returns rows from the first table where a match exists.

Q15. Difference between ANTI JOIN and LEFT JOIN?

LEFT JOIN returns all rows from the left table with matching right table data.

ANTI JOIN returns only rows from the left table where no match exists in the right table.

In SQL, ANTI JOIN can be written as:

LEFT JOIN + WHERE right_table.key IS NULL

or:

NOT EXISTS

Q16. Why should we avoid NOT IN when NULL exists?

If the subquery contains NULL, **NOT IN** can return unexpected or empty results.

Use **NOT EXISTS** or **LEFT JOIN + IS NULL** instead.

Q17. What is NATURAL JOIN?

NATURAL JOIN automatically joins tables based on columns with the same name.

Q18. Why is NATURAL JOIN avoided in production?

Because join columns are selected automatically.

If table structure changes, output can change unexpectedly.

Explicit JOIN with ON condition is safer.

15. Final Revision

FULL OUTER JOIN → complete comparison from both sides

CROSS JOIN → all possible combinations

SELF JOIN → same table joined with itself

NON-EQUI JOIN → range-based join

SEMI JOIN → records where match exists

ANTI JOIN → records where match does not exist

NATURAL JOIN → automatic join on same column names, usually avoided

Final point:

Do not learn joins only by syntax.

Always connect the join with a real business scenario.

That is what helps in interviews.